😎

JAVA

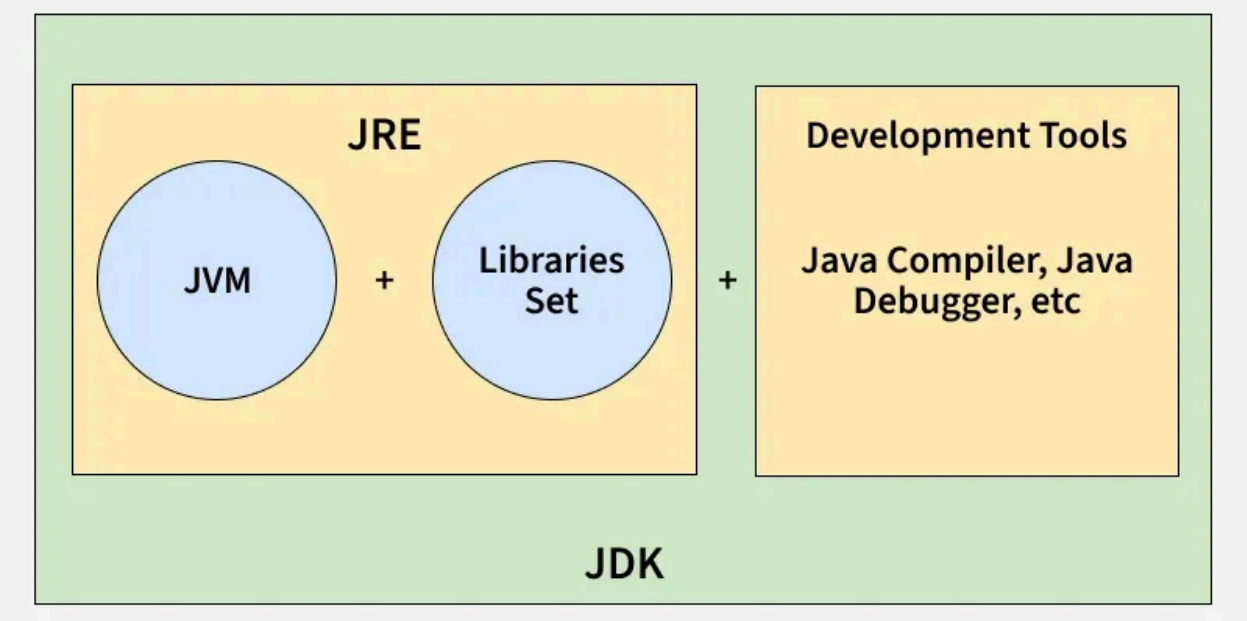
🔰 JAVA – Beginner to Advanced Notes

1️⃣ What is Java?

- Java is a **high-level, object-oriented, platform-independent** programming language.
- Developed by **Sun Microsystems (1995)** → now owned by **Oracle**.
- Motto: **"Write Once, Run Anywhere" (WORA)**.

Why Java?

- Platform independent
- Secure & robust
- Huge community & libraries
- Used in **Web, Android, Backend, Cloud, Big Data**



2️⃣ Flow:

```
.java (sourcecode)
   ↓ javac
.class (bytecode)
   ↓ JVM
Machine Code
```

Components:

- **JDK** → Java Development Kit (compiler + tools)
- **JRE** → Java Runtime Environment (libraries + JVM)
- **JVM** → Java Virtual Machine (runs bytecode)

3️⃣ Basic Java Program

```
classMain {
publicstaticvoidmain(String[] args) {
        System.out.println("Hello Java");
    }
```

```
}
```

Keywords:

- `class` → blueprint
- `main()` → entry point
- `System.out.println()` → print output

4️⃣ Variables & Data Types

Variables

```
int a=10;
```

Primitive Data Types

Type	Size
byte	1 byte
short	2 bytes
int	4 bytes
long	8 bytes
float	4 bytes
double	8 bytes
char	2 bytes
boolean	1 bit

Non-Primitive

- String
- Array
- Class

- Interface

5️⃣ Operators

- Arithmetic → `+ - * / %`
- Relational → `== != > < >= <=`
- Logical → `&& || !`
- Assignment → `= += -=`
- Unary → `++ --`

6️⃣ Control Statements

If-Else

```
if(age >=18){
   System.out.println("Adult");}
else{
   System.out.println("Minor");
}
```

Switch

```
switch(day) {
case1: System.out.println("Mon");break;
default: System.out.println("Invalid");
}
```

7️⃣ Loops

For

```
for(int i=1;i<=5;i++)
    System.out.println(i);
```

While

```
while(i<=5) {
    i++;
}
```

Do-While

```
do {
    i++;
}while(i<=5);
```

8️⃣ Arrays

```
int[] arr = {1,2,3,4};
System.out.println(arr[0]);
```

2D Array

```
int[][] mat =newint[2][2];
```

9️⃣ String in Java

```
Strings="Java";
```

Important Methods

- `length()`
- `charAt()`
- `substring()`
- `equals()`
- `toUpperCase()`

String vs StringBuilder

String	StringBuilder
Immutable	Mutable
Slower	Faster

🔟 Methods (Functions)

```
staticintadd(int a,int b) {
return a+b;
}
```

Method Overloading

```
add(int,int)
add(double,double)
```

OOPS

1️⃣ What is OOP?

OOP = Object Oriented Programming

It is a programming style based on:

- **Objects** (real-world entities)
- **Classes** (blueprints)

📌 Java is **100% object-oriented** (except primitive types).

2️⃣ Class & Object (FOUNDATION)

🔷 Class

A **blueprint** that defines:

- Variables (data)
- Methods (behavior)

```
class Car {
int speed;

void drive() {
        System.out.println("Car is driving");
    }
}
```

🔷 Object

A **real instance** of a class.

```
Car c=new Car();
c.speed =100;
c.drive();
```

3️⃣ Four Pillars of OOP (MOST IMPORTANT ⭐⭐⭐⭐)

🔐 1. Encapsulation

Binding data + methods together & hiding data

Why?

- Security
- Control access
- Maintainability

How?

- `private` variables
- `public` getters/setters

```
classStudent {
privateint age;

public void setAge(int age) {
if(age >0)
this.age = age;
    }

public int getAge() {
return age;
    }
}
```

📌 **Real-life analogy**: ATM → You don't access balance directly.

🧬 2. Inheritance

One class acquires properties of another

```
class Animal {
void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
void bark() {
        System.out.println("Barking");
    }
}
```

Types of Inheritance in Java

Type	Supported
Single	✅
Multilevel	✅
Hierarchical	✅
Multiple (class)	❌
Multiple (interface)	✅

📌 Java avoids **diamond problem**

🎭 3. Polymorphism

One method, many behaviors

🔷 Compile-Time Polymorphism (Method Overloading)

```
classMathUtil {
int add(int a,int b) {
return a + b;
    }
```

```
double add(double a,double b) {
return a + b;
   }
}
```

✔️ Same method name

✔️ Different parameters

🔷 Runtime Polymorphism (Method Overriding)

```
class Parent {
void show() {
      System.out.println("Parent");
   }
}

class Child extends Parent {
@Override
void show() {
      System.out.println("Child");
   }
}
```

```
Parentp=newChild();
p.show();// Child
```

📌 Decision happens **at runtime**

📌 Uses **Dynamic Method Dispatch**

🎯 4. Abstraction

Hiding implementation, showing functionality

🔷 Abstract Class

```
abstract class Shape {
abstract void draw();
}
```

```
class Circle extends Shape {
void draw() {
        System.out.println("Drawing Circle");
    }
}
```

🔷 Interface (MOST IMPORTANT)

```
interface Payment {
void pay();
}
```

```
class UPI implements Payment {
public void pay() {
        System.out.println("UPI Payment");
    }
}
```

📌 Interfaces support **multiple inheritance**

4️⃣ Abstract Class vs Interface (INTERVIEW TABLE ⭐⭐⭐)

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Abstract + default/static
Variables	Any	public static final
Inheritance	Single	Multiple
Constructor	Yes	No

5️⃣ `this` Keyword (VERY COMMON)

Uses:

1. Refer current object
2. Resolve variable conflict
3. Call current constructor

```
class Student {
int age;

    Student(int age) {
this.age = age;
    }
}
```

6️⃣ `super` Keyword

Used to:

- Access parent variable
- Call parent method
- Call parent constructor

```
super.show();
super();
```

7️⃣ final Keyword (VERY IMPORTANT)

Usage	Meaning
final variable	Constant
final method	Cannot override
final class	Cannot inherit

```
final int PI=3;
```

8️⃣ Constructor (OBJECT CREATION)

```
class Student {
    Student() {
        System.out.println("Object created");
    }
}
```

Types:

- Default
- Parameterized
- Copy (manual)

9️⃣ Object Class (ROOT CLASS)

All classes inherit from `Object`

Common methods:

```
toString()
equals()
hashCode()
```

📌 Used heavily in **Collections & HashMap**

🔟 Association, Aggregation & Composition (ADVANCED)

Association

```
class Driver {
   Car car;
}
```

Aggregation (Weak relation)

```
class Library {
   Book book;
}
```

Composition (Strong relation)

```
class House {
Room room=new Room();
}
```

🧠 MOST ASKED QUESTIONS

✅ Why Java doesn’t support multiple inheritance?
→ Diamond problem

✅ Can we override static methods?
→ ❌ No (method hiding)

✅ Can constructor be overridden?
→ ❌ No

✅ Why use abstraction?
→ Security + flexibility

❓ *Is SOLID mandatory?*

✔️ No, but **following it makes code professional & scalable**

❓ *Where is SOLID used?*

✔️ Backend systems, APIs, Microservices, Spring Boot

🔷 What is Object-Oriented Programming?

Interview Answer:

> Object-Oriented Programming is a programming paradigm where we design software using objects that represent real-world entities.
>
> It helps in structuring code using classes and objects, and improves reusability, maintainability, and scalability.

🔷 What are the four pillars of OOP?

Interview Answer:

> The four pillars of OOP are Encapsulation, Inheritance, Polymorphism, and Abstraction.
>
> These principles help in data protection, code reuse, flexibility, and hiding implementation details.

🔷 Explain Encapsulation

Interview Answer:

> Encapsulation means binding data and methods together and restricting direct access to data.
>
> In Java, it is achieved using private variables and public getter and setter methods.
>
> It improves security and controlled access to data.

🔷 Explain Inheritance

Interview Answer:

> Inheritance allows one class to acquire the properties and behavior of another class.
>
> It helps in code reuse and supports hierarchical relationships.
>
> Java supports single, multilevel, and hierarchical inheritance using classes.

🔷 Explain Polymorphism

Interview Answer:

> Polymorphism means one method behaving in multiple ways.
>
> Java supports compile-time polymorphism through method overloading and runtime polymorphism through method overriding.
>
> Runtime polymorphism is achieved using dynamic method dispatch.

🔷 Explain Abstraction

Interview Answer:

> Abstraction is the process of hiding internal implementation details and showing only essential functionality to the user.
>
> In Java, abstraction is achieved using abstract classes and interfaces.
>
> It improves flexibility and reduces system complexity.

🔷 What are SOLID principles?

Interview Answer:

> SOLID principles are a set of five object-oriented design principles that help in writing clean, maintainable, and scalable code.
>
> They are widely used in enterprise-level applications and backend systems.

❓ Is Java fully object-oriented?

Answer:

> Java is almost fully object-oriented, except for primitive data types.

❓ Difference between abstract class and interface?

Answer:

> Abstract class supports partial abstraction and single inheritance, while interface supports full abstraction and multiple inheritance.

🔥 SOLID Principles — Explained Simply (with Java Examples)

🟢 What is SOLID?

SOLID = set of 5 design principles proposed by **Robert C. Martin (Uncle Bob)** to make OOP code:

✔️ Easy to maintain

✔️ Easy to extend

✔️ Less buggy

✔️ Flexible to change

1️⃣ S — Single Responsibility Principle (SRP)

📌 Definition

> A class should have only ONE reason to change

Meaning:

- One class → one job
- Don’t mix responsibilities

❌ Bad Example

```
classStudent {
voidsaveToDatabase() {}
voidcalculateMarks() {}
voidprintReport() {}
}
```

🚫 Problems:

- Database logic + business logic + printing in one class

✅ Good Example

```
classStudent {
voidcalculateMarks() {}
}

classStudentRepository {
voidsaveToDatabase() {}
}

classReportPrinter {
voidprintReport() {}
}
```

✔️ Each class has **one responsibility**

📌 **Interview line**:

> SRP reduces complexity and improves maintainability.

2️⃣ O — Open/Closed Principle (OCP)

📌 Definition

> Classes should be open for extension but closed for modification

Meaning:

- Don’t change existing code
- Add new behavior by extending

❌ Bad Example

```
classPayment {
voidpay(String type) {
if(type.equals("UPI")) {}
elseif(type.equals("CARD")) {}
   }
}
```

🚫 Every new payment → modify code

✅ Good Example (Using Abstraction)

```
interfacePayment {
voidpay();
}

classUPIimplementsPayment {
```

```
publicvoidpay() {
        System.out.println("UPI payment");
    }
}

classCardimplementsPayment {
publicvoidpay() {
        System.out.println("Card payment");
    }
}
```

✔️ Add new payment → **new class**, no modification

📌 **Interview line**:

> OCP prevents breaking existing functionality.

3️⃣ L — Liskov Substitution Principle (LSP)

📌 Definition

> Child class should be replaceable by parent class without breaking the program

---

❌ Bad Example

```
classBird {
voidfly() {}
}

classOstrichextendsBird {
voidfly() {
thrownewRuntimeException("Can't fly");
    }
```

```
}
```

🚫 Ostrich breaks behavior

✅ Good Example

```
classBird {}

interfaceFlyable {
voidfly();
}

classSparrowextendsBirdimplementsFlyable {
publicvoidfly() {}
}
```

✔️ No unexpected behavior

📌 **Interview line**:

> LSP ensures correctness in inheritance.

4️⃣ I — Interface Segregation Principle (ISP)

📌 Definition

> Clients should not be forced to implement methods they don’t use

---

❌ Bad Example

```
interfaceWorker {
voidwork();
voideat();
```

```
}
```

```
classRobotimplementsWorker {
publicvoideat() {}// not needed
}
```

🚫 Robot forced to implement eat()

✅ Good Example

```
interfaceWorkable {
voidwork();
}

interfaceEatable {
voideat();
}

classHumanimplementsWorkable, Eatable {
publicvoidwork() {}
publicvoideat() {}
}

classRobotimplementsWorkable {
publicvoidwork() {}
}
```

✔️ Clean & flexible

📌 **Interview line**:

> ISP avoids unnecessary dependencies.

5️⃣ D — Dependency Inversion Principle (DIP)

📌 Definition

> High-level modules should not depend on low-level modules.
> Both should depend on abstractions.

❌ Bad Example

```
classKeyboard {}
classComputer {
Keyboardk=newKeyboard();
}
```

🚫 Tightly coupled

✅ Good Example

```
interfaceInputDevice {}

classKeyboardimplementsInputDevice {}

classComputer {
    InputDevice device;

    Computer(InputDevice device) {
this.device = device;
    }
}
```

✔️ Loose coupling

✔️ Easy to replace device

📌 **Interview line**:

> DIP improves flexibility and testability.